

管理者によるお知らせ登録・編集用画面機能

バックエンド側の作成・編集

すでにエンティティ `Announcement.java` とリポジトリ `AnnouncementRepository.java` は存在しているので、これらに対応する形で **Service層** と **Controller層** のソースコードを組み立てていきます。

中でも `isDeletable` カラム（0:削除不可, 1:削除可能）のバリデーションや、新規登録（UUID自動生成）と編集（上書き）をスマートに統合するロジックを肉付けしているところが今回の特徴です。

Service層の作成 (`AnnouncementService.java`)

新規登録時は、渡された `announcementId` が空（新規）であるかをジャッジし、空であれば Java 側で安全に UUID を生成してセットします。

既存の ID がある場合は、DB から呼び出して上書き（編集）処理を行います。

```
package com.appspace.backend.service;

import com.appspace.backend.entity.Announcement;
import com.appspace.backend.repository.AnnouncementRepository;
```

```

import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.UUID;

@Service
@RequiredArgsConstructor
@Transactional
public class AnnouncementService {

    private final AnnouncementRepository announcementRepository;

    /**
     * □ジック1: お知らせ一覧を全件取得する
     */
    public List<Announcement> getAllAnnouncements() {
        return announcementRepository.findAll();
    }

    /**
     * □ジック2: お知らせの【新規登録】および【編集】を保存する
     * @param id 新規の場合は null または空文字、編集の場合は対象の UUID
     */
    public Announcement saveOrUpdateAnnouncement(String id, String title, String about, Integer isDeletable) {
        Announcement announcement;

        // IDの有無によって「新規登録」か「上書き編集」かをジャッジします
        if (id != null && !id.trim().isEmpty()) {
            // 【編集の場合】既存レコードをリポジトリから掘り起こす
            announcement = announcementRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("指定されたお知らせが見つかりません。ID: " + id));
        }
    }
}

```

```

    } else {
        // 【新規登録の場合】新しくインスタンスを立ち上げ、UUIDを払い出す
        announcement = new Announcement();
        announcement.setAnnouncementId(UUID.randomUUID().toString());
    }

    // 既存フィールド名に綺麗にマッピングして格納
    announcement.setAnnouncementTitle(title.trim());
    announcement.setAnnouncementAbout(about.trim());

    // データベースへ保存（JPAがINSERTかUPDATEかを自動で判別します）
    Announcement savedAnnouncement = announcementRepository.save(announcement);
    System.out.println("=== [Service] お知らせを保存しました (ID: " + savedAnnouncement.getAnnouncementId() + ") ===");

    return savedAnnouncement;
}
}

```

Controller層の作成 (AdminAnnouncementController.java)

管理者のコントロールパネル（Vue側）から届くリクエストのオブジェクトを受け取り、バックエンド側の二重防壁（空文字バリデーション）を通したのち、サービスへとトラフィックを流します。

```

package com.appspace.backend.controller;

import com.appspace.backend.entity.Announcement;
import com.appspace.backend.service.AnnouncementService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/admin/announcements")
@RequiredArgsConstructor
@CrossOrigin(origins = "https://*.app.github.dev")
public class AdminAnnouncementController {

    private final AnnouncementService announcementService;

    /**
     * 窓口1: 管理者用に現在登録されているお知らせ一覧を全件取得する
     * GET http://localhost:8080/api/admin/announcements
     */
    @GetMapping
    public ResponseEntity<List<Announcement>> getAllAnnouncements() {
        return ResponseEntity.ok(announcementService.getAllAnnouncements());
    }

    /**
     * 窓口2: お知らせの新規登録・上書き編集リクエストをまとめて受け付ける
     * POST http://localhost:8080/api/admin/announcements/save
     */
    @PostMapping("/save")
    public ResponseEntity<String> saveAnnouncement(@RequestBody Map<String, Object> payload) {
        // フィールド名に適合させてリクエストを取り出し
        String id = (String) payload.get("announcementId"); // 新規のときは null または ""
        String title = (String) payload.get("announcementTitle");
        String about = (String) payload.get("announcementAbout");
    }
}

```

```
// バックエンド側での不正データブロック（二重防壁バリデーション）
if (title == null || title.trim().isEmpty()) {
    return ResponseEntity.badRequest().body("【却下】 お知らせのタイトルを入力してください。");
}
if (about == null || about.trim().isEmpty()) {
    return ResponseEntity.badRequest().body("【却下】 お知らせの本文・詳細を入力してください。");
}

try {
    // サービス層の保存・編集ロジックをキック
    announcementService.saveOrUpdateAnnouncement(id, title, about, 1);
    return ResponseEntity.ok("お知らせの内容を正常に反映・保存しました。");
} catch (Exception e) {
    return ResponseEntity.internalServerError().body("お知らせの保存処理中にエラーが発生しました: " + e.getMessage());
}
}
```

SecurityConfig.java へのアクセス許可追加

新設した管理者専用のエンドポイントが、Spring Security によって拒否されないよう、以前と同様にセキュリティ設定ファイルの許可リスト（`requestMatchers`）へ以下のようにパスを追記してください。

`"/api/admin/announcements/**", // ★既存の Announcement 管理用パスを許可リストに追加`

フロントエンド側の実装

今回は既存の「統括管理者用 コントロールパネル」に滑り込ませるための、お知らせ管理専用のコンポーネント `AdminAnnouncementPanel.vue` を新設し、親画面のタブにドッキングさせる手順を解説します。

フロントエンド： `AdminAnnouncementPanel.vue` の実装

`src/components/`（または管理者用コンポーネントを取りまとめているディレクトリ）に `AdminAnnouncementPanel.vue` を新規作成し、以下のコードを記述してください。

※ 登録と編集のフォーム状態をスマートに切り替えられるUXを盛り込んでいます。

```
<template>
  <div class="admin-announcement-panel">
    <div class="panel-header">
      <h3>お知らせ（インフォメーション） 管理一覧</h3>
      <button
        @click="prepareNewAnnouncement"
```

```
        class="btn-primary"
        v-if="!isEditing"
    >
        新しいお知らせを作成
    </button>
</div>

<div v-if="isEditing" class="form-container-box">
    <h4>{{ form.announcementId ? 'お知らせの編集' : '新しいお知らせの登録' }}</h4>

    <div class="form-group">
        <label>タイトル <span class="required">※必須</span></label>
        <input
            v-model="form.announcementTitle"
            type="text"
            placeholder="例： 年末年始の稼働日に関するお知らせ"
            maxlength="100"
        />
    </div>
</div>
```



```
<div class="form-group">
  <label>本文・詳細内容 <span class="required">※必須</span></label>
  <textarea
    v-model="form.announcementAbout"
    rows="5"
    placeholder="一般ユーザーのダッシュボード最上部に掲示される文章を入力してください。"
  ></textarea>
</div>

<div class="form-actions">
  <button @click="handleSave" class="btn-save">内容を保存する</button>
  <button @click="cancelEdit" class="btn-cancel">キャンセル</button>
</div>
</div>

<div class="announcement-list" v-if="announcements.length > 0">
  <div
    v-for="item in announcements"
    :key="item.announcementId"
    class="announcement-card"
  >
    <div class="card-title-bar">
      <h5>{{ item.announcementTitle }}</h5>
    </div>
    <p class="card-content-text">{{ item.announcementAbout }}</p>
  </div>
</div>
```

```

    <div class="card-footer">
      <small class="id-text">ID: {{ item.announcementId }}</small>
      <button @click="startEdit(item)" class="btn-edit-trigger">✎ 編集する</button>
    </div>
  </div>
</div>

<div v-else-if="!isEditing" class="no-data-alert">
  現在登録されているお知らせはありません。右上のボタンから作成してください。🍰
</div>
</div>
</template>

<script setup>
import { ref, onMounted, watch } from 'vue';
import apiClient from '../api'; // ご利用の共通APIクライアント

// 状態管理変数
const announcements = ref([]); // お知らせリスト
const isEditing = ref(false); // フォームを開いているかどうかのフラグ

// フォームの入力データを束ねるオブジェクト（既存のJavaフィールド名に準拠）
const form = ref({
  announcementId: '',
  announcementTitle: '',
  announcementAbout: ''
});

```

```
// 1. サーバーから最新のお知らせ一覧を全件引っ張ってくる関数
const fetchAnnouncements = async () => {
  try {
    const response = await apiClient.get('/admin/announcements');
    announcements.value = response.data;
  } catch (error) {
    console.error('お知らせ一覧の取得に失敗しました:', error);
    alert('お知らせ一覧の読み込みに失敗しました。');
  }
};
```

```
// 新規作成フォームの準備
const prepareNewAnnouncement = () => {
  form.value = {
    announcementId: '',
    announcementTitle: '',
    announcementAbout: '',
  };
  isEditing.value = true;
};
```

// 2. 「編集する」ボタンが押されたとき、フォームに既存の値をロードする関数

```
const startEdit = (item) => {  
  form.value = { ...item };  
  isEditing.value = true;  
  // 編集しやすいように画面上部へスクロール  
  window.scrollTo({ top: 0, behavior: 'smooth' });  
};
```

// キャンセル処理

```
const cancelEdit = () => {  
  isEditing.value = false;  
};
```

// 3. 「内容を保存する」ボタンが押されたときの処理（新規・上書き共通）

```
const handleSave = async () => {  
  // フロントエンド防壁（簡易バリデーション）  
  if (!form.value.announcementTitle.trim()) {  
    alert('タイトルを入力してください。');  
    return;  
  }  
  if (!form.value.announcementAbout.trim()) {  
    alert('本文・詳細内容を入力してください。');  
    return;  
  }  
}
```

```
try {
  // バックエンドの POST /api/admin/announcements/save へ送信
  const response = await apiClient.post('/admin/announcements/save', form.value);
  alert(response.data); // 「お知らせの内容を正常に反映・保存しました」を表示
  isEditing.value = false;
  fetchAnnouncements(); // 一覧をパッと再ロード
} catch (error) {
  console.error('お知らせ保存エラー:', error);
  alert(error.response?.data || 'お知らせの保存に失敗しました。');
}
};

// 画面表示時に自動ロード
onMounted(() => {
  fetchAnnouncements();
});
</script>

<style scoped>
.admin-announcement-panel { padding: 5px; text-align: left; font-family: sans-serif; }
.panel-header { display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
  border-bottom: 1px solid #e0e0e0;
  padding-bottom: 10px;
}
.panel-header h3 { margin: 0; color: #2c3e50; font-size: 1.2rem; }
```

```
/* ボタン系のスタイル */
.btn-primary {
  background-color: #0288d1;
  color: white;
  border: none;
  padding: 8px 16px;
  border-radius: 4px;
  font-weight: bold;
  cursor: pointer;
}
.btn-primary:hover { background-color: #01579b; }

/* フォームのスタイリング */
.form-container-box { background: #f8f9fa;
  border: 1px solid #b0bec5;
  border-radius: 6px;
  padding: 20px;
  margin-bottom: 25px;
  box-shadow: inset 0 2px 4px rgba(0,0,0,0.02);
}
.form-container-box h4 { margin-top: 0; margin-bottom: 15px; color: #37474f; border-left: 4px solid #0288d1; padding-left: 10px; }
.form-group { margin-bottom: 15px; }
.form-group label { display: block; font-weight: bold; font-size: 0.9rem; color: #455a64; margin-bottom: 6px; }
.form-group input[type="text"], .form-group textarea { width: 100%;
  padding: 10px;
  border: 1px solid #cfd8dc;
  border-radius: 4px;
  box-sizing: border-box;
  font-size: 0.95rem;
}
.required { color: #d32f2f; }
```

```
.form-actions { display: flex; gap: 10px; }
.btn-save {
  background-color: #2e7d32;
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: 4px;
  font-weight: bold;
  cursor: pointer;
}
.btn-save:hover { background-color: #1b5e20; }
.btn-cancel { background-color: #cfd8dc; color: #37474f; border: none; padding: 10px 16px; border-radius: 4px; cursor: pointer; }
.btn-cancel:hover { background-color: #b0bec5; }

/* お知らせカードのスタイリング */
.announcement-list {
  display: flex;
  flex-direction: column;
  gap: 15px;
  margin-top: 15px;
}
.announcement-card {
  background: #ffffff;
  border: 1px solid #cfd8dc;
  border-radius: 6px;
  padding: 15px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.04);
}
```

```
.card-title-bar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  margin-bottom: 10px;  
}  
.card-title-bar h5 {  
  margin: 0;  
  font-size: 1.05rem;  
  color: #263238;  
  font-weight: bold;  
}  
  
.card-content-text {  
  font-size: 0.95rem;  
  color: #455a64;  
  margin: 0 0 12px 0;  
  line-height: 1.5;  
  white-space: pre-wrap;  
}  
  
.card-footer {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  border-top: 1px dashed #eceff1;  
  padding-top: 10px;  
}
```



```
.id-text { color: #90a4ae; font-family: monospace; }
.btn-edit-trigger { background: transparent;
  border: 1px solid #0288d1;
  color: #0288d1;
  padding: 4px 12px;
  border-radius: 4px;
  font-size: 0.85rem;
  font-weight: bold;
  cursor: pointer;
  transition: all 0.2s;
}
.btn-edit-trigger:hover { background: #e1f5fe; }

.no-data-alert {
  background: #eceff1;
  border: 1px solid #b0bec5;
  text-align: center;
  padding: 25px;
  border-radius: 6px;
  color: #546e7a;
  font-weight: bold;
}
</style>
```

「統括管理者用 コントロールパネル」 メイン画面への統合

新設した `AdminAnnouncementPanel.vue` を、既存のコントロールパネル（前回の「打刻申請確認」タブを追加したファイル）へ組み込みます。

今回は「打刻モード一括制御」タブのさらに後ろに「お知らせ管理」タブを配置します。

メインのコントロールパネル画面の修正イメージ

```
<template>
  <div class="admin-control-panel">
    <h2>統括管理者用 コントロールパネル</h2>

    <div class="tabs-header">
      <button
        @click="currentTab = 'accounts'"
        :class="{ active: currentTab === 'accounts' }"
      >
        アカウント一覧・退会管理
      </button>
```

```
<button
  @click="currentTab = 'requests'"
  :class="{ active: currentTab === 'requests' }"
>
  打刻申請確認
</button>

<button
  @click="currentTab = 'modes'"
  :class="{ active: currentTab === 'modes' }"
>
  打刻モード一括制御
</button>

<button
  @click="currentTab = 'announcements'"
  :class="{ active: currentTab === 'announcements' }"
>
  お知らせ管理
</button>
</div>

<div class="tab-content-body">
  <div v-if="currentTab === 'accounts'">
    <!-- 'accounts'に対応するエリアの記述 -->
  </div>
```

```
<div v-if="currentTab === 'requests'">
  <AdminAttendanceApprovalView />
</div>

<div v-if="currentTab === 'modes'">
  <!-- 'modes'に対応するエリアの記述 -->
</div>

<div v-if="currentTab === 'announcements'">
  <AdminAnnouncementPanel />
</div>
</div>
</div>
</template>

<script setup>
import { ref } from 'vue';
import AdminAttendanceApprovalView from './AdminAttendanceApprovalView.vue';
// 今回作成した新コンポーネントをインポート
import AdminAnnouncementPanel from './AdminAnnouncementPanel.vue';

/* これ以降のscriptタグの記述は前回と同じ */
</script>

<style scoped>
/* 前回のCSSスタイルをそのまま維持してください */
</style>
```

連動の確認テスト手順

ファイルを配置後、ブラウザでコントロールパネルにアクセスし、以下の手順で往復トラフィックを確認してみましょう。

- **新規作成テスト:**

- 新しく設置された「お知らせ管理」タブをポチッと押します。
- 「新しいお知らせを作成」ボタンを押し、タイトルと本文を入力、「保存する」を押します。
- 「正常に保存しました」とアラートが出て、リストの一番下（または上）に**UUIDが割り振られたカードがパッと自動生成**されれば大成功です！

- **上書き編集テスト:**

- 先ほど作ったカードの「編集する」ボタンを押します。
- フォームに先ほど書いたタイトルや本文がシュッと自動でロードされることを確認し、一部の文章を書き換えて「保存する」を押します。
- 既存のカードの中身が**新しい文章へ綺麗に上書き更新**されているか確認します（データベース側でも新しい行が作られず、同じUUIDの行がUPDATEされていれば完璧です）。